Gymnasium Oberwil

Maturarbeit

Bildsynthese von menschlichen Gesichtern mithilfe von Kuenstlicher Intelligenz

Eine Anwendung eines Autoencoders

Luis Wirth, 4a

Betreut von
Dr. Jonas Gloor

Abgegeben im September 2019

Inhaltsverzeichnis

1 Vorwort **2**

2 Einleitung **3**

3 Konventionen und Notation **4**

4 Machinelles Lernen - Theorie **5**
4.1 Allgemeine Begriffe . . . 5
4.1.1 Daten . . . 5
4.1.2 Modelle . . . 5
4.2 Training . . . 6
4.2.1 Kostenfunktionen . . . 6
4.2.2 Gradientenverfahren . . . 7
4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning . . . 8
4.2.4 Adam Optimizer . . . 9
4.3 Künstliche Neuronale Netze und Deep Learning . . . 10
4.3.1 Perzeptron . . . 10
4.3.2 Biologische Analogie . . . 11
4.3.3 Erweiterung der Kuenstlichen Neuronen . . . 11
4.3.4 Topologie der Kuenstlichen Neuronalen Netzen . . . 12
4.3.5 Vorwaertspropagierung . . . 13
4.3.6 Gewichte und Neigungen initalisieren* . . . 15
4.3.7 Rueckwaertspropagierung . . . 15
4.3.8 Universal Approximation Theorem . . . 16
4.4 Autoencoder und Convolutional KNN . . . 16
4.4.1 Autoencoder . . . 16
4.4.2 Bedeutung von Autoencodern . . . 16
4.4.3 Convolutional KNN . . . 16
4.4.4 Variational Autoencoder? . . . 16
4.4.5 Detangled Autoencoder? . . . 16

5 Entwicklung eines Autoencoders **17**
5.1 Tools . . . 17
5.1.1 Python . . . 17
5.1.2 Tensorflow . . . 17
5.1.3 Keras . . . 17
5.2 Idee . . . 17
5.3 Voraussetzungen . . . 17
5.3.1 Trainingsdaten . . . 17
5.4 Programmieren . . . 17

6 Diskussion? **18**

7 Schlusswort **19**

1 Vorwort

2 Einleitung

Die Leitfrage lautet: **Wie können mithilfe von Machine Learning Bilder von künstlichen meschlichen Gesichtern generiert werden?**

3 Konventionen und Notation

Beschriftung

$\mathbf{v}$	ein Vektor Test
$\mathbf{M}$	eine Matrix
$\mathbf{M}^{\mathrm{T}}$	eine transponierte Matrix

Konventionen

- Falls ein (Zeilen- oder Spalten-) Vektor $\mathbf{v}$ als Argument in eine Funktion f gegeben wird, bedeutet dies, dass die einzelnen Komponenten des Vektors die Parameter der Funktion sind.

$$f(\mathbf{v}) = f(v_1, \ldots, v_n)$$

- Falls die Klammern, in denen der Vektor $\mathbf{v}$ steht, als eckige Klammern [] geschrieben sind, bedeutet dies, dass es sich um die Vektorisierung einer Funktion f handelt, somit wird die Funktion auf jede Vektorkomponente angewandt.

$$f[\mathbf{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

ERWEITERN

4 Machinelles Lernen - Theorie

Der Themenbereich des **Machinellen Lernens** beschäftigt sich mit Algorithmen und mathematischen Modellen, welche von selber lernen, Probleme zu loesen. Hierzu werden keine expliziten Instruktionen fest einprogrammiert (engl.: hard coding), sondern das Modell wird trainiert und optimiert sich von selbst. Dabei wird das Modell mit Trainingsdaten befuettert, mithilfe eine Korrelation zwischen den Inputdaten und den Outputdaten erlernt werden soll. Der Zusammenhang der Daten wird von den Modellen verallgemeinert und das Modell kann dann Vorhersagen fuer neue Daten machen.

4.1 Allgemeine Begriffe

4.1.1 Daten

Der **Trainingsdatensatz** $\mathbb{T}$ beinhaltet gewisse **Attribute**, die Merkmalstypen der Inputdaten. Die **Features** sind dann die spezifischen Werte der Attribute eines Trainingssample. Die sogenannten **Labels** sind die erwarteten Outputs, welche der Algorithmus vorherzusagen hat. Diese Vorhersagen werden mit den Labels abgeglichen und so bewertet. Anhand der Bewertungen wird dann eine Optimierung des Modells vorgenommen. Unter korrekten Bedingungen (keine Ueberanpassung) findet kein Auswendiglernen der Trainingsdaten statt, sondern ein Generalisieren des Zusammenhangs anhand von Mustern und Gesetzmassigkeiten.

Um eine endgueltige Bewertung des Modells durchzufuehren, wird ein Testdatensatz, welcher nicht Teil des Trainingsdatensatz ist, genutzt um Vorhersagen zu machen. Dies garantiert, dass kein Auswendiglernen moeglich ist.

Die Inputs/Features werden in einem Vektor $\mathbf{x} = (x_1, x_2, \ldots, x_n)^{\mathrm{T}}$ und die Outputs/Vohersagen in einem Vektor $\mathbf{y} = (y_1, y_2, \ldots, y_m)^{\mathrm{T}}$ zusammengefasst. Das gleiche macht man mit den Labels, wobei diese mit $\hat{\mathbf{y}} = (\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_m)^{\mathrm{T}}$ bezeichnet werden.

BESSERS BEISPIEL -> UM SPAETER KONKRETES MODELL ZU ERKLAEREN Am Beispiel eines Datensatz, welcher fuer Bilderkennung von Hunden und Katzen genutzt wird, werden diese Begriffe verdeutlicht. Die Inputdaten sind die Fotos von Hunden und Katzen. Deren Attribute sind die moeglichen RGB-Farbenwerte (drei Zahlen, welche von 0 bis 255 reichen) aller Pixel. Die Features sind die spezifischen RGB-Werte der Pixel fuer die einzelnen Bilder. Die Outputdaten sind die Labels, welche in diesem Fall angeben, ob ein Hund oder eine Katze auf dem Bild gezeigt wird. Dies wird meistens mit einer One-Hot-Kodierung bzw. einem One-Hot-Vektor gemacht. Jeder Labelart wird eine eigene Vektorkomponente zugewiesen. Falls das Label aktiv ist, ist die Komponent 1 ansonsten 0. Somit kann Hund als $\mathbf{l}_H = (1, 0)^T$ reprasentiert werden und Katze als $\mathbf{l}_K = (0, 1)^T$.

Abbildung 1: eine Veranschaulichung des beschriebenen Machine Learning Modells

4.1.2 Modelle

Ein **Modell** ist eine mathematische Funktion $f\colon \mathbb{R}^n \to \mathbb{R}^m$, welche die Inputs auf die Outputs abbildet $\mathbf{y} = f(\mathbf{x})$. Im Zusammenhang mit Wertevorhersagen spricht man von einem Regressionsmodell. Das Verhalten dieses Modells wird bestimmt durch seine **Modellparameter** $\phi_1, \phi_2, \ldots$. Das Ziel ist es, die Modellparameter so einzustellen, dass die Vorhersagen $\mathbf{y}$ besser mit den Labels $\hat{\mathbf{y}}$ uebereinstimmen. Dies wird iterativ gemacht, indem immer wieder leichte Anpassungen an den Parametern vorgenommen werden, bis sie das gewuenschte Resultat liefern.

Das wohl einfachste Regressionsmodell ist eine Regressionsgerade. Diese ist angemessen, falls ein einfacher linearen Zusammenhang der Form $y = \phi_1 x + \phi_0$ zwischen den Features und den Labels besteht. Nun muessten nur noch ϕ_0 und ϕ_1 bestimmt werden.
Neben den gelernten Parametern, gibt es auch noch sogenannte **Hyperparameter**. Diese koennen nicht erlernt werden, sondern muessen manuell vor dem Training gewaehlt werden und koennen den Lernvorgang erheblich beeinflussen.

Fuer Machine Learning haben sich gewisse Modelle besonders guet etabliert, darunter waeren: Support Vector Machines, Evolutionäre Algorithmen, und Künstliche Neuronale Netze. Diese Arbeit wird sich vorwiegend mit Neuronalen Netzen auseinandersetzen.

BEISPIELMODELL GLEICHES WIE VORHIN

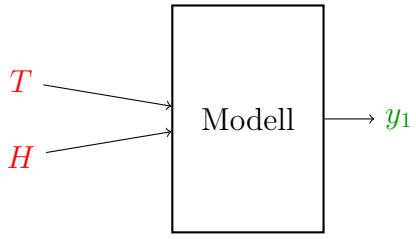

Abbildung 2: Darstellung eines Modells

4.2 Training

4.2.1 Kostenfunktionen

Einsicht ist der erste Schritt zur Besserung. Das gilt auch beim Machine Learning, deshalb muss beim Training zuerst die Genauigkeit des Modells bewertet werden. Dafuer gibt es sogennante **Kosten-, Verlust- oder Fehlerfunktionen**. Sie sollen ein Mass fuer die Abweichung des Outputs $\mathbf{y}$ vom Label $\hat{\mathbf{y}}$ sein.

Die Fehler der einzelnen Outputs $c(y_i, \hat{y}_i)$ werden aufsummiert um den Fehler $C(\mathbf{y}_j, \hat{\mathbf{y}}_j)$ (wobei $\mathbf{y}_j$ die Vorhersagen sind und $\hat{\mathbf{y}}_j$ die Labels sind) einer gesamten Vorhersage zu berechnen. Siehe Gleichung (1).
Der Fehler $\bar{C}(\mathbb{T})$ eines ganzen Datensatzes $\mathbb{T}$, der Groesse p ergibt sich aus dem arithmetischen Mittel der einzelnen Vorhersagen-Fehler. Siehe Gleichung (2).

$$C(\mathbf{y}, \hat{\mathbf{y}}) = \sum_{i=1}^{m} c(y_i, \hat{y}_i) \qquad (1) \qquad\qquad \bar{C}(\mathbb{T}) = \frac{1}{p} \sum_{j=1}^{p} C(\mathbf{y}_j, \hat{\mathbf{y}}_j) \qquad (2)$$

Eine Kostenfunktion sollte folgende Eigenschaften aufweisen:

- c ist minimal, wenn $y = \hat{y}$
- c waechst mit $|\mathbf{y} - \hat{\mathbf{y}}|$
- C ist nach jedem y_n partiell differenzierbar (erklaert in Sektion (**??**))

Eine beliebte Kostenfunktion ist die "Mittlere quadratische Abweichung" (engl.: mean squared error).
$C_{MSE} = \frac{1}{2n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{2n} (\hat{\mathbf{y}} - \mathbf{y})^2$. Sie erfuellt alle anforderungen, denn: Sie ist 0 falls $y = \hat{y}$. Sie ist proportional zu $(\hat{y} - y)^2$ und ihre Ableitung nach y lautet: $C' = \frac{1}{n}(y - \hat{y})$

4.2.2 Gradientenverfahren

Um ein Modell nun zu trainieren, muss man verstehen, dass es sich um ein Optimierungsproblem handelt. Das Modell ist am besten, also macht die besten Vorraussagen, wenn die Funktionswerte der Fehlerfunktion am kleinsten sind. Deshalb muss diese Fehlerfunktion C minimiert werden. Hierbei muss die Funktion C nicht mehr in Abhaengigkeit der Inputs betrachtet werden, sondern in Abhaengigkeit von den Modellparametern $C(\phi_1, \phi_2, \ldots, \phi_n)$, denn diese sollen angepasst werden, um das Modell zu verbessern. Fuer diese Optimiertung wird das sogennante **Gradientenverfahren** (engl.: Gradient descent) verwendet. [1]

Allgemein koennen mithilfe des Gradientenverfahrens Funkionen $f(x_1, x_2, \ldots, x_n)$ vom Typ $\mathbb{R}^n \to \mathbb{R}$ (wie z.B. die Fehlerfunktion) minimiert werden. Dies geschieht, indem ein Startpunkt (Ortsvektor) $\mathbf{p}_0$, dessen Komponenten den Input von f darstellt, gewaehlt wird. Nun werden iterativ neue Punkte $\mathbf{p_z}$ gesucht, welche immer naeher beim lokalen Minimun liegen, also Punkte, die den Funktionswert $f(\mathbf{p}_z)$ immer kleiner werden lassen. Dies wird durchgeführt, bis der Punkt genug nahe beim lokalen Minimun ist.

Dafuer muss ein Vektor $\mathbf{b}_z$ bestimmt werden, welcher auf den Punkt $\mathbf{p}_z$ draufaddiert einen neuen Punkt $\mathbf{p}_{z+1}$ bildet, bei dem der Funktionswert $f(\mathbf{p}_{z+1})$ kleiner ist als der von $f(\mathbf{p}_z)$. Dies geschieht am effizientesten, wenn $\mathbf{b}_z$ in die Richtung der staerksten Funktionswertabnahme zeigt.
Hierzu braucht man den sogennanten **Gradient** ∇, wofür man wiederum partielle Ableitungen braucht.

Partielle Ableitungen

Partielle Ableitungen sind eine Erweiterung der "normalen" Ableitungen auf multidimensionale Funktionen $f(x_1, \cdots, x_n)$. Man leitet dabei nur nach einem Parameter x_i ab und betrachtet die restlichen Argumente als Konstanten. Es gelten die gleichen Ableitungsregeln wie bei der nicht-partiellen Ableitung. Die partielle Ableitung einer Funktion $f(x_1, \cdots, x_n)$ bezueglich der Variable x_i in einem Punkt $\mathbf{a} = (a_1, \cdots, a_n)^T$ ist analog zur "normalen" Ableitung folgendermassen definiert:

$$\frac{\partial f}{\partial x_i}(\mathbf{a}) := \lim_{h \to 0} \frac{f(a_1, \cdots, a_i + h, \cdots, a_n) - f(a_1, \cdots, a_i, \cdots, a_n)}{h}$$

Geometisch ist dies die Steigung der Tangente im Punkt $\mathbf{a}$ in Richtung der Achse des Parametern x_i, nach dem man ableitet.

[1] Im Gymnasium wird beigebracht die lokalen Extrema (inkl. lokalen Minimas) zu bestimmen, indem die erste Ableitung f' gebildet wird und gleich null gesetzt wird. Dies ist hier nicht moeglich, da die Funktion $C'(\phi_1, \phi_2, \ldots, \phi_n)$ deutlich zu komplex ist, um die Nullstellen zu bestimmen. Deshalb wird das Gradientenverfahren verwendet.

Gradient

Der Gradient ∇ ist ein Differentialoperator, welcher ein Skalarfeld auf ein Vektorfeld (das sogennante Gradientenfeld) abbildet.
Dabei fasst man alle partiellen Ableitungen einer Funktion f in einem Vektor zusammen.

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Geometisch ist der Gradient $\nabla f(\mathbf{p})$ einer Funktion f in einem Punkt $\mathbf{p}$ dann der Vektor, welcher in die Richtung des steilsten Anstiegs von f zeigt. Sein Betrag gibt die Staerke des Anstiegs an.

Da der Gradient in die Richtung des steilsten Anstiegs zeigt, sollte der Vektor $\mathbf{b}_z$ in die Richtung des negierten Gradient der Funktion f im Punkt $\mathbf{p}_z$ zeigen. Also kann jetzt das iterative Annahern an das lokalen Minimums so beschrieben werden (wobei η eine spaeter erklaerte Schrittgroesse darstellt):

$$\mathbf{p}_{z+1} = \mathbf{p}_z - \eta \cdot \nabla f(\mathbf{p}_z) \tag{3}$$

BEISPIEL FUER GRADIENTENVERFAHREN MIT ABBILDUNG

Abbildung 3: Darstellung des Gradientenabstiegs

Waehrend des Gradientenverfahren konvergiert der Punkt $\mathbf{p}_z$ zu einem belibigen *lokalen* Minimum, abhängig davon wie der Startpunkt $\mathbf{p}_0$ gewaehlt wurde. Da dieser meist zufaellig bestimmt wird, ist es eine Glückssache ein sehr niedriges Minimum zu finden.

Die sogennante **Lernrate** η aus Gleichung (3) ist ein Hyperparameter. Sie ist ein positiver Proportionalitatsfaktor, welcher die Schrittgroesse des Gradientenabstieg bestimmt. Sie muss je nach zu minimierender Funktion anderst gewaehlt werden. Dabei hilft nur ausporbieren. Falls η nicht gut gewahelt wurde, gibt es Probleme beim Training:

- Falls η zu klein ist, verlaueft das Trainings unnotig langsam und braucht sehr lange. Ausserdem kann es passieren, dass man bei einem hohen lokalen Minimum stecken bleibt.
- Falls η zu gross ist, passiert es, dass man ueber das lokale Minimum hinaus schiesst und nur darum herum springt.

4.2.3 Stochastisches Gradientenverfahren fuer Machine Learning

Wie vorhin erklaert, wird fuer das Trainieren eines Modells das Gradientenverfahren benutzt. Konkret wird die Kostenfunktion $C(\mathbf{y}, \hat{\mathbf{y}}|\phi_0, \ldots, \phi_n)$ (ϕ sind die Modellparameter, $\mathbf{y}$ die Vorhersagen und $\hat{\mathbf{y}}$ die Labels) minimieren, indem die Parameter angepasst werden. Dadurch macht das Modell immer bessere Vorhersagen. Fuer nur eine Iteration des Gradientenverfahren, muesste man den Gradienten fuer den *gesamten* Trainingsdatensatz berechnen. Dies waere zwar ein exakter Prozess, aber ein extrem langsamer zugleich. Bei grossen Datensaetzen wuerde es eine Ewigkeit dauern, bis das Modell nur annahernd guete Vorhersagen machen wuerde.

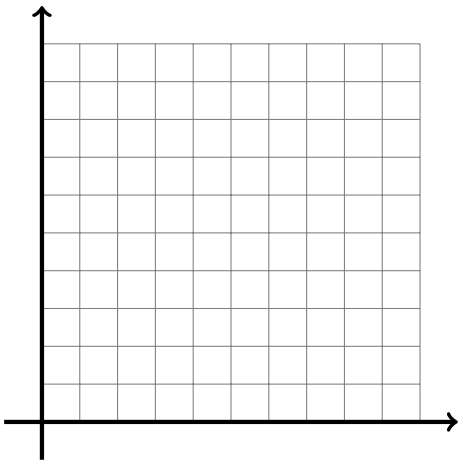

Abbildung 4: Visualiesierung verschiedener Lernraten

Aus diesem Grund verwendet man eine leicht abgeänderte Variante dieses Verfahren, naemlich das **Stochastische Gradientenverfahren** (engl.: Stochastic Gradient Descent). Hierfuer wird der "echte" Gradient des gesamten Datensatzes mit dem Gradienten einiger Trainingsbeispiel approximiert. Dazu wird der Trainingsdatensatz in sogennante **Mini-Batches** eingeteilt und der Gradient jeweils pro Mini-Batch berechnet. Als Konsequenz finden deutlich mehr Iterationen statt in *einer* Durchkaemmung der Trainingsdaten, welche man als **Epoche** bezeichnet. Oft wird mehrere Epochen lang trainiert. Der Gradient eines genug grossen Mini-Batches ist zwar nicht ganz exakt, aber approximiert den Gradieten des gesamten Datensatzen genuegend gut. Sowohl die Mini-Batch Groesse, wie auch die Anzahl Epochen sind weitere Hyperparameter.

Die partiellen Ableitungen der gesamten Trainingsdaten wird mit dem arithmetischen Mittel der partiellen Ableitungen eines Mini-Batches der Groesse q approximiert. Siehe Gleichung (4).

$$\frac{\partial \bar{C}}{\partial \phi_k} \approx \frac{1}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \phi_k} \tag{4}$$

Eine Iteration des Stochastischen Graidentenverfahren wird analog zu Gleichung (3) folgendermassen durchgefuehrt:

$$\phi_{k,t+1} = \phi_{k,t} - \frac{\eta}{q} \sum_{i=1}^{q} \frac{\partial C_i}{\partial \phi_{k,t}} \tag{5}$$

4.2.4 Adam Optimizer

ZUSAMMENFASSUNG TRAININGSALGORITHMUS

4.3 Künstliche Neuronale Netze und Deep Learning

Das wohl beste Modell fuer die meisten Problemstellungen in Bereich Machinelles Lernen (Bilderkennung, Spracherkennung, etc.) ist das **Kuenstliche Neuronale Netz**, kurz KNN (eng.: Neural Network). Die Methoden welche benutzt werden um Neuronale Netze zu traniert fasst man unter dem Begriff **Deep Learning** zusammen.
Kuenstliche Neuronale Netze sind zum Teil biologisch inspiriert von Nervensystemen von Lebewesen. Sie sind aber lediglich eine Abstraktion der Informationverarbeitung und versuchen nicht eine moeglichst genaue biologische Abbildung darzustellen. Es gibt nicht nur eine Art von Neuralem Netz, sondern es exisitieren die verschiedensten Architekturen, welche je nach Problemstellung ausgewaehlt werden muessen. Diese Arbeit wird sich spaeter vorallem mit sogennanten Autoencodern auseinandersetzen.

4.3.1 Perzeptron

Um den Aufbau und die Funktion eines Kuenstlichen Neuronalen Netz besser zu verstehen, wird im folgenden ein Vorgaenger des KNN erklaert: das **Perzeptron**.

Das einlagige Perzeptron wurde erstmals 1958 von Frank Rosenblatt vorgestellt. Dieses besteht aus einem einzigen Kuenstlichen Neuron. Dieses Kuenstliche Neuron hat mehrere binaere Inputs und einen einzigen binaeren Output. Binaer bedeutet, dass der Wert nur entweder 0 (*aus*) oder 1 (*ein*) sein kann. Des weiteren besitzt es mehrere sogenannte **Gewichte** $w_1, \ldots, w_n \in \mathbb{R}$, fuer jeden Input ein Gewicht. Diese sind reelle Zahlen, welche das Verhalten des Perzeptron bestimmen. Die **gewichtete Summe**, also die Summe aller Produkte der Inputs mit ihrem Gewicht, wird mit $\tilde{z}$ bezeichnet. Sie ist das gleiche wie das Skalarprodukt des Gewichtevektor mit dem Inputvektor: $\tilde{z} = \sum_{i=1}^{n} w_i x_i = \mathbf{w} \cdot \mathbf{x}$
Zusaetzlich besitzt das Perzeptron einen **Schwellenwert** $\tilde{b}$. Zusammen mit den Gewichten, bildet er die Modellparameter. Das Perzeptron verhaelt sich so, dass falls die gewichtete Summe groesser als der Schwellwert ist, das Neuron feuert, d.h. der Output betraegt 1. Andernfalls ist er 0. Siehe erster Teil der Gleichung (6). Es ist gaengig die Ungleichung der Bedingung in die Nullstellenform zu bringen und $\tilde{b}$ durch die **Neigung** (engl.: Bias) $b = -\tilde{b}$ zu ersetzten. Somit lautet die Ungleichung: $\tilde{z} + b > 0$. Der Term $\tilde{z} + b$ wird mit z bezeichnet. Siehe Rest der Gleichung (6). Die Neigung gibt an wie stark das Neuron dazu neigt zu feuern. Ein hohes b laesst ein Neuron auch trotz einigen Nullen in den Inputs feuern, waehrend es fuer ein tiefes b nicht feuern lassen wuerde.

$$p(\mathbf{x}) = \begin{cases} 1 & \text{falls } \tilde{z} > \tilde{b} \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \tilde{z} + b > 0 \\ 0 & \text{ansonsten} \end{cases} = \begin{cases} 1 & \text{falls } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{ansonsten} \end{cases} \tag{6}$$

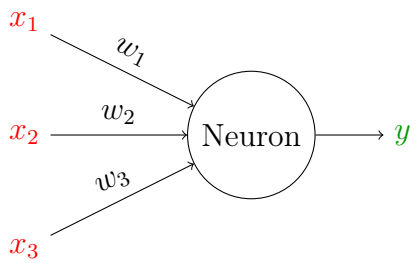

Abbildung 5: ein Perzeptron mit drei Inputs

Fuer das Trainieren des Perzeptron existieren spezielle Verfahren, welche hier aber nicht relevant sind. Das Gradientenverfahren kann naemlich nicht verwendet werden. Der Grund dafuer sollte spaeter in Sektion (4.3.3) einleuchtend werden.

Nun stellt sich die Frage, was ein Perzeptron erlernen kann und wofuer es genutzt werden kann. Das Perzeptron ist lediglich ein **Linearer Klassifikator** der Form $y = w_1x_1 + \cdots + w_nx_n$. Es kann die Features in zwei Klassen 0 oder 1 einordnen. Ueberschreitet y den Schwellenwert, werden die Features der Klasse 1 zugeordnet, sonst der Klasse 0. Jedoch muessen diese Klassen linear separierbar sein. Das bedeutet, dass die Inputvektoren $\mathbf{x}$ durch eine Hyperebene trennbar sein muessen. Bei zwei Features (zwei Inputs) handelt es sich bei der Hyperebene um eine Gerade welche die Vektoren auftrennt. Siehe Abbildung (6). Fuer drei Featuers ist es eine Ebene. ZU ABSTRAKT -> BEISPIEL
Eine Funktionen, welche nicht linear separierbar ist (z.B XOR-Verknuepfungen), kann von einem Perzeptron nicht erlernt werden.

Abbildung 6: erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts)

(**wiki:perzeptron**)

4.3.2 Biologische Analogie

SCHREIBEN

4.3.3 Erweiterung der Kuenstlichen Neuronen

Ein Perzeptron ist, wie vorhin erklaert, nur in der Lage, lineare Klassifikationen durchzufuehren. Um nun auch kompliziertere Probleme zu loesen, muss das Prinzip ausgebaut werden. Ausserdem brauchen wir ein Kuenstliches Neuron, welches sich besonders gut als Baustein fuer KNNs eignet.

Kuenstliche Neuronen im Allgemeinen Kuenstliche Neuronen sind immer so aufgebaut, dass sie einen oder mehrere Inputs haben und einen einzigen Output. Zu jedem Input x_i ist ein Gewicht w_{ji} assoziert. Zuerst wird die gewichtete Summe der Inputs $\tilde{z}$ gebildet. Die Neigung b wird ebenfalls draufaddiert, um z zu erhalten. Nun muss die sogennante Aktivierung a gebildet werden. Sie entspricht dem Output des Neurons. Die Aktivierung a ist das Resultat der Aktivierungsfunktion φ angewendet auf z. $a = \varphi(z)$. Die verschiedenen Kuenstlichen Neuronen unterscheiden sich fast nur in ihrer Aktivierungsfunktion.

Perzeptronen als Kuenstliche Neuronen Zuerst noch einmal ein Blick auf das Perzeptron in Betracht der Aktivierungsfunktion. Ein wesentlicher Unterschied des Perzeptron gegenueber sonstigen Kuenstlichen Neuronen, besteht darin, dass seine Inputs und Outputs nur binaere Werte annehmen koennen. Um dieses Verhalten des Perzeptron zu erhalten, muss eine Stufenfunktion als Aktivierungfunktion verwendet werden: die Heaviside-Funktion Θ. Sie hat einen einzigen Stufensprung bei $x = 0$ vom Wert 0 auf 1. Siehe Abb. (7).

Stueckweise lineare Neuronen Der naechste Schritt nach dem binaeren Perzeptron sind sogenannte stueckweise lineare Neuronen. Sie verwenden eine stueckweise lineare Aktivierungsfunktion. Diese bildet ein beschraenktes Intervall linear ab. Die Werte ausserhalb werden auf die konstanten Werte 0 oder 1 abbgebildet. Siehe Abb. (8).

Die Inputs koennen jetzt beliebige reelle Zahlen sein (vorzugsweise in der Naehe von 0 und 1). Ein KNN aus ausschliesslich linearen Neuronen ist ziemlich nutzlos, da jedliche Verkettung von

$$\varphi^{\text{hlim}}(z) = \Theta(z) = \begin{cases} 1 & \text{falls } z \geq 0 \\ 0 & \text{falls } z < 0 \end{cases}$$

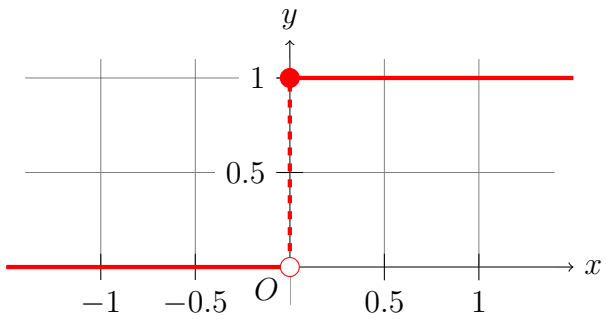

Abbildung 7: die Definition und der Graph der Heaviside-Funktion Θ

linearen Funktion auch durch eine einzige lineare Funktion dargestellt werden koennte. Somit hat ein KNN gegenueber einem einzelnen Neuron keinen Mehrwert. Ausserdem koennen mit diesen Neuron auch nur lineare Probleme geloest werden.

$$\varphi^{\text{pwl}}(z) = \begin{cases} 1 & \text{falls } z > \frac{1}{2} \\ z + \frac{1}{2} & \text{falls } -\frac{1}{2} < z < \frac{1}{2} \\ 0 & \text{falls } z < \frac{1}{2} \end{cases}$$

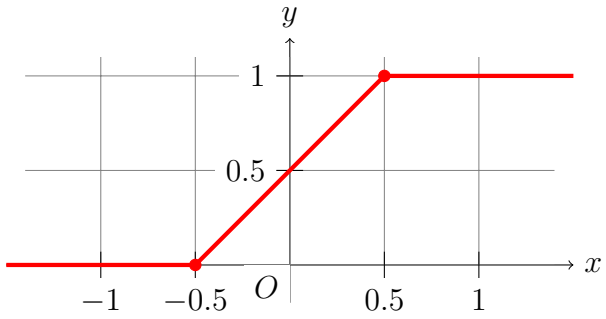

Abbildung 8: die Formel und der Graph einer stueckweisen linearen Funktion

Sigmoide Neuronen Die logische naechste Erweiterung sind nicht-lineare Neuronen. Ein sehr beliebter Kandidat dafuer sind Sigmoide Neuronen. Den Namen haben sie von ihrer Aktivierungsfunktion: der Sigmoidfunktion σ. Eine sehr wichtige Eigenschaft von ihr ist, dass sie - im Gegensatz zu den vorhin genannten Aktivierungsfunktion - ueberall differenzierbar und strikt monoton steigend ist. Erst fuer diese Aktivierungsfunktion, kann das Gradientenverfahren angewendet werden und somit das KNN trainiert werden. Die anderen Funktionen haben Stellen bei welchen die Ableitungsfunktion null betraegt, somit kann das lokale Minimum nicht gefunden werden. Nicht nur die Sigmoidfunktion erfuellt diese Bedingungen, sondern auch andere. Die Sigmoidfunktion wird bevorzugt, da ihre Ableitung sehr simpel ist, siehe Abb. (9). Die Nicht-linearitaet dieser Neuronen ermoeglicht ein Erlernen von deutlich komplexeren Sachverhalten. Und wie spaeter in Sektion (4.3.8) erlautert, ist es moeglich mit der Kompostion von nicht-linearen Funktionen jede belibige Funktion zu approximieren!

Die Simgmoid-Funktion besitzt eine einzige Wendestelle bei $x = 0$ und hat zwei Asymptoten, eine fuer $x \to -\infty$ mit $y = 0$ und eine zweite fuer $x \to \infty$ mit $y = 1$. Siehe Abb. (9).

4.3.4 Topologie der Kuenstlichen Neuronalen Netzen

Nun sollten diese Sigmoiden-Neuronen als Bausteine verwendet werden, um ein Kuenstliches Neuronales Netz zu bilden. Dazu werden sie miteinander verbundet und bilden so ein Netz, aehnlich wie ein Nervensystem. Diese Neuronen sind in verschieden Schichten (engl.: Layers) arangiert. Die erste ist die **Inputschicht**. Sie behinhalten die Inputneuronen. Dies sind eigentlich keine richtigen Neuronen sondern eher Platzhalter fuer ihren jeweiligen Inputwert x_i. Als letztes kommt die **Outputschicht** mit den Outputneuronen, welche jeweils einen Outputwert

$$\varphi^{\text{sig}}(z) = \sigma(z) = \frac{1}{1+e^{-z}}$$
$$\sigma'(z) = \sigma(z)(1-\sigma(z))$$

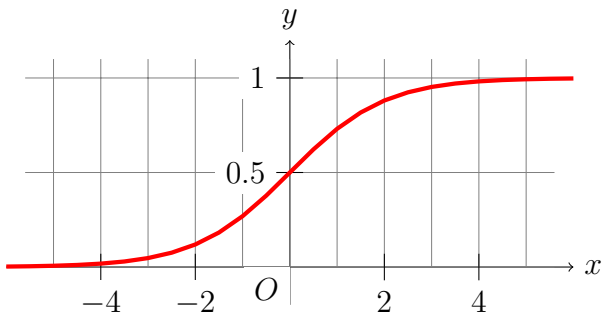

Abbildung 9: die Formel und der Graph der Sigmoid-Funktion σ

y_i besitzen. Dazwischen liegen die **Zwischenschichten** (engl.: Hiddenlayer). Von ihnen kann es beliebig viele geben und in ihnen beliebig viele Neuronen. Der Aufbau eines KNN bezeichnet man als **Topologie** des Netzes, es handelt sich dabei um Hyperparameter.

Jedes Neuron aus einer Schicht ist mit jedem Neuron aus der naechsten Schicht ueber Verbindungen gekoppelt. Alle Verbindungen besitzten ein Gewicht analog zum Perzeptron. Die Aktivierung, also der Output, eines Neurons wandert entlang den jeweiligen Verbindung zu allen Neuronen der naechsten Schicht und dient als deren Input. Die soeben beschriebe Art von KNN nennt man **Feedforward-Netz**, da alle Werte ausschliesslich nach vorne propagiert werden.

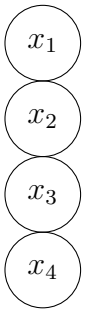

Abbildung 10: eine Darstellung der Schichtung eines KNN

4.3.5 Vorwaertspropagierung

Nachdem nun der Aufbau eines KNNs erklaert wurde, sollte nun die mathematische Funktionsweise des Modells erklaert werden. Hierfuer muessen einige Konventionen zur Bezeichnung der Teile eines KNNs getroffen werden:

- n_j^l bezeichnet das j-te Neuron in der l-ten Schicht.
- z_j^l ist die gewichtete Summe der Inputs des j-ten Neuron in der l-ten Schicht.
- a_j^l ist die Aktivierung des j-ten Neurons in der l-ten Schicht.
- b_j^l ist die Neigung des j-ten Neuron in der l-ten Schicht.
- $w_{j,k}^l$ ist das Gewicht der Verbindung vom k-ten Neuron in der $(l-1)$-ten Schicht zum j-ten Neuron in der l-ten Schicht (man beachte die Reihenfolge) [2].

[2] Diese Konvention scheint auf den ersten Blick unintuitiv, macht jedoch Sinn fuer die spaeteren mathematischen Operationen mit Matrizen in Sektion (4.3.7).

- L soll die gesamte Anzahl der Schichten sein.
- φ ist die gewaehlte Aktivierungsfunktion (grundsaetzlich ist diese immer die Sigmoidfunktion σ).

Die Vorwaertspropagierung beginnt bei den Inputneuronen, welche mit ihren Inputwerten befuettert werden. Diese werden fuer eine konsistente Nomenklatur gleich wie die Aktivierungen der anderen Neuronen mit a_j^0 bezeichnet, wobei j der Index des Neurons ist.
Die restlichen Aktivierungen der Neuronen werden rekursiv anhand der Aktivierungen der voherigen Schicht berechnet (siehe Gleichung (8)).
Zuerst wird die gewichtete Summe der Aktivierungen a_k^{l-1} der vorherigen Schicht mit den assozierten Gewichten $w_{j,k}^l$ gebildet. Dazu gehoert auch die jeweilige Neigung b_j^l. Auf diese Summe wird dann die Aktivierungsfunktion φ angewandt. Das ist dann die Aktivierung des Neurons.

$$z_j^l = \sum_k w_{j,k}^l a_k^{l-1} + b_j^l \tag{7}$$

$$a_j^l = \varphi\left(\sum_k w_{j,k}^l a_k^{l-1} + b_j^l\right) = \varphi\left(z_j^l\right) \tag{8}$$

Da KNNs weit verbreitet sind in der Bilderkennung und Spracherkennung, ist es nicht unueblich, dass diese sehr viele Neuronen und Verbindungen (ueber 100'000) besitzen. Um hierbei nicht den Ueberblick zu verlieren und um nicht in den Indizes zu ertrinken, greift man auf **Lineare Algebra** zurueck. Man verwendet Matrizen und Vektoren um die vielen Variabeln zusammen zu fassen. Ausserdem koennen Computer deutlich schneller Matrixoperationen ausfuehren, als alle Zahlen einzeln zu verrechnen. Dies beschleunigt das Training der Modelle um ein Vielfaches. Dies wird spaeter in Sektion (**??**) noch mehr erlauetert.

Um nun lineare Algebra ins Spiel zu bringen, wird eine **Gewichtsmatrix** $\mathbf{W}^l$ fuer jede Schicht l definiert. Die Eintraege dieser Matrix sind einfach die Gewichte welche zu den Neuronen der l-ten Schicht verlaufen. Das heisst der Eintrag in der j-ten Zeile und in der k-ten Spalte ist $w_{j,k}^l$ (Siehe Gl. (9)). Ebenfalls werden alle Neigung b_j^l in einer Schicht l zu einem Vektor $\mathbf{b}^l$ zusammmen gefasst. Und zu guter Letzt wird ein Aktivierungsvektor $\mathbf{a}^l$, welcher alle Aktiverungen a_j^l einer Schicht l einfaengt.

$$\mathbf{W}^l = \begin{pmatrix} w_{1,1}^l & w_{1,2}^l & \cdots & w_{1,n}^l \\ w_{2,1}^l & w_{2,2}^l & \cdots & w_{2,n}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1}^l & w_{m,2}^l & \cdots & w_{m,n}^l \end{pmatrix} \tag{9}$$

Um nun Gleichung (8) in der Matrixform zu schreiben, muss noch die Vektoriesierung einer Funktion eingefuehrt werden.

Vektoriesierung einer Funktion

Die Vektoriserung einer Funktion f, geschrieben als $f[\mathbf{v}]$ ist eine neue Funktion, welche die Funktion f auf jede Vektorkomponente anwendet und einen neuen Vektor als Rueckgabewert hat.

$$f[\mathbf{v}] = \begin{pmatrix} f(v_1) \\ \vdots \\ f(v_n) \end{pmatrix}$$

Nun kann die Matrixschreibweise eingefuehrt werden (siehe Gleichung (11)).
Zuerst wird eine Matrixmultiplikation der Gewichtsmatrix $\mathbf{W}^l$ mit dem Aktivierungsvektor $\mathbf{a}^{l-1}$ der vorherigen Schicht durchgefuehrt. Darauf wird der Neigunsvektor addiert. Und zu guter Letzt wird die vektorisierte Aktivierungsfunktion φ angewandt.

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l \tag{10}$$

$$\mathbf{a}^l = \varphi\left[\mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{b}^l\right] = \varphi\left[\mathbf{z}^l\right] \tag{11}$$

4.3.6 Gewichte und Neigungen initalisieren*

4.3.7 Rueckwaertspropagierung

Das Gradientenverfahren fuer KNN bezeichnet man als **Rueckwaertspropagierung** (engl.: Backpropagation).
Das Ziel dieses Algorithmus ist es die beiden Partiellen Ableitungen $\frac{\partial C}{\partial w^l_{j,k}}$ und $\frac{\partial C}{\partial b^l_j}$ zu bestimmen.
Hierfuer ist es sinnvoll eine Zwischenwert, welchen wir Fehler δ^l_j nennen.

Hadamard-Produkt

Das Hadamard-Produkt ist ein spezielles Produkt zweier gleichgrossen Matrizen. Die resultierende Matrix ergibt sich aus der elementweisen Multiplikation der Ausgangsmatrizen.

$$\mathbf{A} \odot \mathbf{B} = \begin{pmatrix} a_{1,1}b_{1,1} & \cdots & a_{1,n}b_{1,n} \\ \vdots & \ddots & \vdots \\ a_{m,1}b_{m,1} & \cdots & a_{m,n}b_{m,n} \end{pmatrix} \in \mathbb{R}^{m \times n} \qquad \mathbf{v} \odot \mathbf{w} = \begin{pmatrix} v_1 w_1 \\ \vdots \\ v_n w_n \end{pmatrix}$$

$$\delta^l_j := \frac{\partial C}{\partial z^l_j} \tag{12}$$

Viel Kettenregel.

$$\frac{\partial C}{\partial w_{j,k}} = \frac{\partial C}{\partial a_k} \frac{\partial a_k}{\partial z_k} \frac{\partial z_k}{\partial w_{j,k}} \tag{13}$$

Die vier magischen Gleichungen:

Der Fehler in der Outputschicht

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L) \tag{BP1}$$

Matrixform

$$\delta^L = \nabla_a C \odot \sigma'(z^L) \tag{BP1a}$$

Fehler anhand des Fehlers der naechsten Schicht

$$\delta^l = ((w^{l+1})^{\mathrm{T}} \delta^{l+1}) \odot \sigma'(z^l) \tag{BP2}$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l \tag{BP3}$$

$$\frac{\partial C}{\partial w_{j,k}^l} = a_k^{l-1} \delta_j^l \tag{BP4}$$

4.3.8 Universal Approximation Theorem

Neural Network kann alles erlernen

4.4 Autoencoder und Convolutional KNN

4.4.1 Autoencoder

4.4.2 Bedeutung von Autoencodern

Datenkompression Hauptkomponentenanalyse und Dimensionalitatsreduktion

Datengenerierung

Eigenschaften erkennen

4.4.3 Convolutional KNN

4.4.4 Variational Autoencoder?

4.4.5 Detangled Autoencoder?

5 Entwicklung eines Autoencoders

5.1 Tools

5.1.1 Python

5.1.2 Tensorflow

5.1.3 Keras

5.2 Idee

5.3 Voraussetzungen

5.3.1 Trainingsdaten

MegaFace Dataset

5.4 Programmieren

6 Diskussion?

7 Schlusswort

Abbildungsverzeichnis

1 eine Veranschaulichung des beschriebenen Machine Learning Modells 5
2 Darstellung eines Modells 6
3 Darstellung des Gradientenabstiegs 8
4 Visualiesierung verschiedener Lernraten 9
5 ein Perzeptron mit drei Inputs 10
6 erfolgreiche lineare Separierung (links) und das Versagen bei XOR (rechts) . . . 11
7 die Definition und der Graph der Heaviside-Funktion Θ 12
8 die Formel und der Graph einer stueckweisen linearen Funktion 12
9 die Formel und der Graph der Sigmoid-Funktion σ 13
10 eine Darstellung der Schichtung eines KNN 13

Tabellenverzeichnis

Selbstständigkeitserklärung

Ich erkläre hiermit, dass ich diese Arbeit selbständig durchgeführt und keine anderen als die angegebene Quellen, Hilfsmittel und Hilfspersonen beigezogen habe. Alle Textstellen in der Arbeit, die wörtlich oder sinngemäss aus Quellen entnommen wurden, habe ich als solche gekennzeichnet.

Luis Wirth